

Corso di Laurea in Ingegneria e Scienze Informatiche

Fancy Title

Tesi di laurea in:
NOME DEL CORSO DEL RELATORE

Relatore

Prof. Supervisor Here

Candidato

Candidate Name

Abstract

Opzionale. Poche righe al massimo.

Contents

Abstract	iii
1 Introduzione	1
1.1 Contesto e motivazione	1
1.2 Obiettivi del progetto	1
1.3 Struttura della tesi	1
2 Background	3
2.1 Sviluppo di applicazioni full stack monolinguaggio	3
2.1.1 Panoramica delle soluzioni esistenti e loro limiti	3
2.1.2 Il portabilità e integrazione con librerie native	3
2.2 Kotlin e framework multitarget come risposta al gap	3
2.2.1 Kotlin Multiplatform	3
2.2.2 Interoperabilità con C tramite cinterop	3
2.3 Lo standard FMI e il formato FMU	3
2.3.1 Functional Mock-up Interface (FMI 2.0)	3
2.3.2 Struttura di un file FMU e modalità operative	3
3 Analisi	5
3.1 Requisiti	5
3.2 Modello del dominio	5
3.2.1 Vincoli	5
4 Design	7
4.1 Architettura	7
4.2 Design di dettaglio	7
4.2.1 Interfacciamento con librerie native	7
4.2.2 Backend multiplatforma	7
4.2.3 Frontend multiplatforma	7

5	Implementazione	9
5.1	Binding C/Kotlin e configurazione del linker per piattaforma	9
5.2	Gestione del ciclo di vita dell'FMU	9
5.3	Estrazione dei metadati e gestione delle variabili	9
5.4	Esecuzione della simulazione	9
5.5	Ricompilazione degli FMU su macOS ARM64	9
5.6	Strumenti di processo: Gradle, CI/CD e pipeline multiplatforma .	9
6	Valutazione	11
6.1	Test unitari e di integrazione	11
6.2	Copertura multiplatforma nella CI	11
7	Conclusioni e Sviluppi Futuri	13
7.1	Limitazioni	13
7.2	Sviluppi futuri	13
7.3	Conclusioni	13
		15
	Bibliography	15

List of Figures

LIST OF FIGURES

List of Listings

LIST OF LISTINGS

Chapter 1

Introduzione

1.1 Contesto e motivazione

1.2 Obiettivi del progetto

1.3 Struttura della tesi

Chapter 2

Background

2.1 Sviluppo di applicazioni full stack monolin- guaggio

[CCO21] [CCSO20]

2.1.1 Panoramica delle soluzioni esistenti e loro limiti

2.1.2 Il portabilità e integrazione con librerie native

2.2 Kotlin e framework multitarget come risposta al gap

2.2.1 Kotlin Multiplatform

2.2.2 Interoperabilità con C tramite cinterop

2.3 Lo standard FMI e il formato FMU

2.3.1 Functional Mock-up Interface (FMI 2.0)

2.3.2 Struttura di un file FMU e modalità operative

Chapter 3

Analisi

3.1 Requisiti

3.2 Modello del dominio

3.2.1 Vincoli

Chapter 4

Design

4.1 Architettura

4.2 Design di dettaglio

4.2.1 Interfacciamento con librerie native

4.2.2 Backend multiplatforma

4.2.3 Frontend multiplatforma

Chapter 5

Implementazione

- 5.1 Binding C/Kotlin e configurazione del linker per piattaforma
- 5.2 Gestione del ciclo di vita dell'FMU
- 5.3 Estrazione dei metadati e gestione delle variabili
- 5.4 Esecuzione della simulazione
- 5.5 Ricompilazione degli FMU su macOS ARM64
- 5.6 Strumenti di processo: Gradle, CI/CD e pipeline multiplatforma

Chapter 6

Valutazione

6.1 Test unitari e di integrazione

6.2 Copertura multiplatforma nella CI

Chapter 7

Conclusioni e Sviluppi Futuri

7.1 Limitazioni

7.2 Sviluppi futuri

7.3 Conclusioni

Bibliography

- [CCO21] Giovanni Ciatto, Roberta Calegari, and Andrea Omicini. Lazy stream manipulation in Prolog via backtracking: The case of 2P-KT. In Wolfgang Faber, Gerhard Friedrich, Martin Gebser, and Michael Morak, editors, *Logics in Artificial Intelligence*, volume 12678 of *Lecture Notes in Computer Science*, pages 407–420. Springer, 2021. 17th European Conference, JELIA 2021, Virtual Event, May 17–20, 2021, Proceedings.
- [CCSO20] Giovanni Ciatto, Davide Calvaresi, Michael I. Schumacher, and Andrea Omicini. An abstract framework for agent-based explanations in AI. In *19th International Conference on Autonomous Agents and MultiAgent Systems*, pages 1816–1818. International Foundation for Autonomous Agents and Multiagent Systems, May 2020. Extended Abstract.